

Week 6 - Inheritance - Polymorphism

1. Develop a program to calculate late fines for library items. The program should utilize a base class 'LibraryItem' with a due and return date and derive two classes 'Book' and 'DVD' to calculate and display the late fine for each type of library item. The fine calculation for 'Book' is based on a flat rate of 1 per day for overdue items. The fine calculation for 'DVD' is based on a flat rate of 2 per day for overdue items.

Input format :

The input consists of two lines of input, where each line contains a date.

The first line represents the due date as a string, and the second line represents the return date as a string.

Output format :

For each library item (Book and DVD), display the following:

Due Date: (Due Date)

Return Date: (Return Date)

Late Fine: (Late Fine)

Sample test cases :

Input 1 :

2023-11-01

2023-11-05

Output 1 :

Due Date: 2023-11-01

Return Date: 2023-11-05

Late Fine: 4

Due Date: 2023-11-01

Return Date: 2023-11-05

Late Fine: 8

Input 2 :

2023-11-15

2023-11-15

Output 2 :

Due Date: 2023-11-15

Return Date: 2023-11-15

Late Fine: 0

Due Date: 2023-11-15

Return Date: 2023-11-15

Late Fine: 0

Program

```
class LibraryItem:
    def __init__(self,dd,rd):
        self.dd=dd
        self.rd=rd
    def display_details(self):
        print(f'Due Date: {self.dd}')
        print(f'Return Date: {self.rd}')

class Book(LibraryItem):
    def __init__(self,dd,rd):
        super().__init__(dd,rd)
    def get_fine(self):
        ans = int(self.rd[-2:]) - int(self.dd[-2:])
        return ans*1

class DVD(LibraryItem):
    def __init__(self,dd,rd):
        super().__init__(dd,rd)

    def get_fine(self):
        ans = int(self.rd[-2:]) - int(self.dd[-2:])
        return ans*2
```

2. Helen wants to create a program for student record management with a class hierarchy.

Help her write a program that utilizes hybrid inheritance using the information below.

A base class: Person holds attributes name and ID.

Another base class: Score contains the student's score.

Two derived classes: UndergraduateStudent and PostgraduateStudent, inherit from both Person and Score classes, calculate and display the grade and results respectively.

For Undergraduate Students, Grade is calculated as:

If the score is 90 or above, the grade is 'A'.

If the score is 80 to 89, the grade is 'B'.

If the score is 70 to 79, the grade is 'C'.

If the score is 60 to 69, the grade is 'D'.

If the score is below 60, the grade is an 'F'.

For Postgraduate Students, Result is calculated as:

If the score is 60 or above, the result is 'Pass'.

If the score is below 60, the result is 'Fail'.

Input format :

The first line of input consists of a string, representing the UG student's name.

The second line consists of a string, representing the UG student's ID.

The third line consists of an integer, representing the UG student's score.

The fourth line consists of a string, representing the PG student's name.

The fifth line consists of a string, representing the PG student's ID.

The sixth line consists of an integer, representing the PG student's score.

Output format :

The first line of output prints the grade of UG student in the format: "Student [UG Student's Name] with ID [UG Student's ID] received grade [UG Student's Grade]."

The second line prints the result of the PG student in the format: "Student [PG Student's Name] with ID [PG Student's ID] [Pass/Fail] the course."

Sample test cases :

Input 1 :

John

CS101

75

Alice

ECE111

55

Output 1 :

Student John with ID CS101 received grade C.

Student Alice with ID ECE111 Fail the course.

Input 2 :

Mary

EEE65

85

David

CSE13

92

Output 2 :

Student Mary with ID EEE65 received grade B.

Student David with ID CSE13 Pass the course.

Input 3 :

Ella
ECE20
45
George
Mech101
30

Output 3 :

Student Ella with ID ECE20 received grade F.
Student George with ID Mech101 Fail the course.

Program

```
class Person:
```

```
    def __init__(self, name, student_id):  
        self.name = name  
        self.student_id = student_id
```

```
class Score:
```

```
    def __init__(self, score):  
        self.score = score
```

```
class UndergraduateStudent(Person, Score):
```

```
    def __init__(self, name, student_id, score):  
        Person.__init__(self, name, student_id)  
        Score.__init__(self, score)
```

```
    def displayGrade(self):
```

```
        if self.score >= 90:  
            grade = 'A'  
        elif self.score >= 80:  
            grade = 'B'  
        elif self.score >= 70:  
            grade = 'C'  
        elif self.score >= 60:  
            grade = 'D'  
        else:
```

```
            grade = 'F'  
        print(f"Student {self.name} with ID {self.student_id} received grade {grade}.")
```

```

class PostgraduateStudent(Person, Score):
    def __init__(self, name, student_id, score):
        Person.__init__(self, name, student_id)
        Score.__init__(self, score)

    def displayResult(self):
        if self.score >= 60:
            result = "Pass"
        else:
            result = "Fail"
        print(f"Student {self.name} with ID {self.student_id} {result} the course.")

```

3. Sneha is working on a project where she needs to manage a collection of vehicles in her system. The vehicles can either be Cars or Bikes, and each vehicle has certain attributes like brand, year of manufacture, and additional attributes specific to the type of vehicle (such as seats for Cars and electric status for Bikes). Help Sneha implement a program that allows her to store and display details of different vehicles. Your task is to create a program that reads user input for a list of vehicles, stores them in a List, and then prints the details of each vehicle.

Input format :

The first line contains an integer *n*, the number of vehicles.

The next *n* lines have four space-separated inputs:

The first input is the type of vehicle, either "Car" or "Bike".

The second input is the brand of the vehicle (a string).

The third input is the year of manufacture (an integer).

For a Car:

The fourth input is the number of seats (an integer).

For a Bike:

The fourth input is a boolean value indicating if the bike is electric (true or false).

Output format :

For each vehicle, the following details are printed:

If the vehicle is a Car, print:

"Brand: <brand>, Year: <year>

Seats: <seats>"

If the vehicle is a Bike, print:

"Brand: <brand>, Year: <year>

Electric: <true> or <false>"

If the vehicle type is not Bike or Car, print the following statement: "Invalid vehicle type. Please enter 'Car' or 'Bike'."

Sample test cases :**Input 1 :**

2

Car Toyota 2022 5

Bike Tesla 2021 true

Output 1 :

Brand: Toyota, Year: 2022

Seats: 5

Brand: Tesla, Year: 2021

Electric: true

Input 2 :

3

Car Honda 2023 4

Bike Yamaha 2022 false

Car BMW 2021 7

Output 2 :

Brand: Honda, Year: 2023

Seats: 4

Brand: Yamaha, Year: 2022

Electric: false

Brand: BMW, Year: 2021

Seats: 7

Input 3 :

1

Bus Ford 2020 9

Output 3 :

Invalid vehicle type. Please enter 'Car' or 'Bike'.

Program

class Vehicle:

def __init__(self, brand, year):

self.brand = brand

self.year = year

class Car(Vehicle):

def __init__(self, brand, year, seats):

super().__init__(brand, year)

self.seats = seats

```
def print_car_details(self):
    print(f"Brand: {self.brand}, Year: {self.year}")
    print(f"Seats: {self.seats}")
```

```
class Bike(Vehicle):
    def __init__(self, brand, year, is_electric):
        super().__init__(brand, year)
        self.is_electric = is_electric

    def print_bike_details(self):
        print(f"Brand: {self.brand}, Year: {self.year}")
        print(f"Electric: {str(self.is_electric).lower()}")
```

4. Rekha is designing a game with three character classes: Warrior, Mage, and Archer. Each class inherits from a base class Character and overrides the `calculate_damage(base_attack, weapon_mult, skill_bonus)` method to compute attack damage uniquely.

Warriors get a bonus if `skill_bonus > 50`, Mages have a different formula and bonus if `skill_bonus > 70`, and Archers get partial `skill_bonus` addition and a bonus if `skill_bonus > 40`.

Help Rekha implement these classes to calculate damage values for multiple characters.

Formulas:

Warrior:

$\text{damage} = (\text{base_attack} \times \text{weapon_mult}) + \text{skill_bonus}$
 If `skill_bonus > 50`, then $\text{damage} = \text{damage} + 20$

Mage:

$\text{damage} = (\text{base_attack} + \text{skill_bonus}) \times \text{weapon_mult}$
 If `skill_bonus > 70`, then $\text{damage} = \text{damage} + 30$

Archer:

$\text{damage} = (\text{base_attack} \times \text{weapon_mult}) + (0.5 \times \text{skill_bonus})$
 If `skill_bonus > 40`, then $\text{damage} = \text{damage} + 15$

Input format :

The first line of input contains an integer `n`, representing the number of characters.

Each of the next `n` lines contains:

`<char_type> <base_attack> <weapon_mult> <skill_bonus>`

where `char_type` is "warrior", "mage", or "archer", and the rest are floats.

Output format :

For each valid character, print the calculated damage rounded to 1 decimal place.

For invalid character types, print: "Invalid"

Sample test cases :**Input 1 :**

3

warrior 100.0 1.5 30.0

mage 80.0 2.0 60.0

archer 90.0 1.2 20.0

Output 1 :

180.0

280.0

118.0

Input 2 :

2

warrior 50.0 1.0 10.0

hand 40.0 1.5 80.0

Output 2 :

60.0

Invalid

Program

```
class Character:
```

```
    def calculate_damage(self, base_attack, weapon_mult, skill_bonus):  
        pass
```

```
class Warrior(Character):
```

```
    def calculate_damage(self, base_attack, weapon_mult, skill_bonus):  
        damage = (base_attack * weapon_mult) + skill_bonus  
        if skill_bonus > 50:  
            damage += 20  
        print(round(damage, 1))
```

```
class Mage(Character):
```

```
    def calculate_damage(self, base_attack, weapon_mult, skill_bonus):  
        damage = (base_attack + skill_bonus) * weapon_mult  
        if skill_bonus > 70:  
            damage += 30  
        print(round(damage, 1))
```



```

class Archer(Character):
    def calculate_damage(self, base_attack, weapon_mult, skill_bonus):
        damage = (base_attack * weapon_mult) + (0.5 * skill_bonus)
        if skill_bonus > 40:
            damage += 15
        print(round(damage, 1))

```

5. Sita is designing a payment system that supports multiple payment methods: CreditCard, PayPal, and Bitcoin. She needs to create a base class called PaymentMethod with a method process_payment(amount) that is overridden in each subclass to process payments differently. The CreditCard class adds a fixed fee of 5 to the amount, PayPal doubles the amount, and Bitcoin adds a fixed fee of 50. Help her implement these classes and process multiple payments as per the input.

Input format :

The first line of input consists of an integer value, n, representing the number of payment transactions.

The next n lines each contain:

<method> <identifier> <amount>

method is a string: "creditcard", "paypal", or "bitcoin"

identifier is a string representing the card number, email, or wallet address accordingly

amount is a float representing the payment amount

Output format :

For each valid payment, print: <MethodName> Paid <total_amount>

where <MethodName> is CreditCard, PayPal, or Bitcoin, and <total_amount> is the processed payment rounded to one decimal place.

For invalid inputs, print: Invalid

Sample test cases :

Input 1 :

3

creditcard 1234567890123456 100.5

paypal user@example.com 200.0

bitcoin 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa 300.0

Output 1 :

CreditCard Paid 105.5

PayPal Paid 400.0

Bitcoin Paid 350.0

Input 2 :

2

creditcard 1111222233334444 50.0

palpay test@mail.com 75.25

Output 2 :

CreditCard Paid 55.0

Invalid

Program

```
class PaymentMethod:
```

```
    def __init__(self, identifier):
```

```
        self.identifier = identifier
```

```
    def process_payment(self, amount):
```

```
        pass
```

```
class CreditCard(PaymentMethod):
```

```
    def process_payment(self, amount):
```

```
        return amount + 5
```

```
class PayPal(PaymentMethod):
```

```
    def process_payment(self, amount):
```

```
        return amount * 2
```

```
class Bitcoin(PaymentMethod):
```

```
    def process_payment(self, amount):
```

```
        return amount + 50
```

week 7 - file handling

1. Bob, a data analyst, requires a program to automate the process of analyzing character frequency in a given text. This program should allow the user to input a string, calculate the frequency of each character within the text, save these character frequencies to a file named "char_frequency.txt," and display the results.

Input format :

The input consists of the string.

Output format :

The first line prints "Character Frequencies:".

The following lines print the character frequency in the format: "X: Y" where X is the character and Y is the count.

Sample test cases :**Input 1 :**

aaabbbccc

Output 1 :

Character Frequencies:

a: 3

b: 3

c: 3

Input 2 :

Data analyst job is challenging and fun

Output 2 :

Character Frequencies:

D: 1

a: 6

t: 2

n: 5

l: 3

y: 1

s: 2

j: 1

o: 1

b: 1

i: 2

c: 1

h: 1

e: 1

g: 2

d: 1

f: 1

u: 1

Input 3 :

"~~~!!!@@@@##\$\$\$\$%%%%%%%%&&&

Output 3 :

Character Frequencies:

": 1

~: 4

!: 3

@: 4

#: 2

\$: 4

?: 4

&: 3

Program

```
def count_char_frequency(file_name):
```

```
    with open(file_name, 'r') as file:
```

```
        data = file.read()
```

```
    d = {}
```

```
    for key in data:
```

```
        if key not in d and key != ' ':
```

```
            d[key] = data.count(key)
```

```
    return d
```

2. Maria is currently working on a utility called "Whitespace Remover" that performs the following tasks:

Accept a string input that may contain alphabetical characters such as uppercase or lowercase, numbers, alphanumeric, and extra white spaces.

Save this input to a file named 'file.txt'.

Read the content from 'file.txt' and display the string without any extra white spaces.

Maria needs your help in framing a program that achieves this functionality. Write a function to assist Maria.

Input format :

The input is a single string containing alphabetic characters, spaces, and extra white spaces. The input string can have a maximum length of 100 characters.

Output format :

The output should display the modified string with extra white spaces removed.

Additionally, the modified string should be saved to a file named 'file.txt'.

Sample test cases :**Input 1 :**

Hello World!

Output 1 :

Hello World!

Input 2 :

A challenging sentence for whitespace removal.

Output 2 :

A challenging sentence for whitespace removal.

Input 3 :

H3ll0 W0rld! W3lc0m3 t0 th3 w0rld 0f c0mpu7ers 4nd pr0gramm1ng.

Output 3 :

H3ll0 W0rld! W3lc0m3 t0 th3 w0rld 0f c0mpu7ers 4nd pr0gramm1ng.

Program

```
user_input = input()
```

```
with open('file.txt', 'w') as file:
```

```
    file.write(user_input)
```

```
with open('file.txt', 'r') as file:
```

```
    content = file.read()
```

```
cleaned_string = " ".join(content.split())
```

```
print(cleaned_string)
```

```
with open('file.txt', 'w') as file:
```

```
    file.write(cleaned_string)
```

3. Alice is developing a program called "Name Sorter" that helps users organize and sort names alphabetically.

The program takes names as input from the user, saves them in a file, and then displays the names in sorted order.

File Name: sorted_names.txt.

Input format :

The input consists of multiple lines, each containing a name represented as a string. To end the input and proceed with sorting, the user can enter 'q'.

Output format :

The output displays the names in alphabetical order, each name on a new line.

Sample test cases :**Input 1 :**

Alice Smith
John Doe
Emma Johnson
q

Output 1 :

Alice Smith
Emma Johnson
John Doe
Input 2 :
David
Oliver
Sophia
q

Output 2 :

David
Oliver
Sophia

Program

```
def sort_and_save_names(names, filename):  
    names.sort()  
    with open(filename, 'w') as file:  
        for name in names:  
            file.write(name + '\n')  
    return names
```

4. In the vibrant realm of Textopia, where characters dance in harmony, you, the Scribble Maestro, are tasked with creating a masterpiece—a simple text editor that captures the essence of words and stores them in a .txt file. The mission is to craft a program that invites users to input their prose, allowing the characters to pirouette

across the digital stage, and then, with a flourish, save this symphony of text to a magical .txt file.

Input format :

The first line of the input is a string representing the text that is needed to be saved.
The second line of the input is a string representing the file name.

Output format :

The output format displays the message 'Text saved successfully to <"file_name.txt">'.

Sample test cases :

Input 1 :

Magical words created by the user!
MagicalWordsCreatedByUser.txt

Output 1 :

Text saved successfully to "MagicalWordsCreatedByUser.txt.txt"

Input 2 :

!@#\$%^&*()_+{}[]
SpecialCharacters

Output 2 :

Text saved successfully to "SpecialCharacters.txt"

Program

```
text = input()
filename = input()
```

```
filename += ".txt"
```

```
with open(filename, 'w') as file:
    file.write(text)
```

```
print(f'Text saved successfully to "{filename}"')
```

5. John is a data analyst who often works with text files. He needs a program that can analyze the contents of a text file and count the number of times a specific character appears in the file.

John wants a simple program that allows him to specify a file and a character to count within that file.

Input format :

The first line of input consists of the file's name to be analyzed.

The second line of the input consists of the string they want to write to the file.

The third line of the input consists of a character to count within the file.

Output format :

The first line of output prints the message "The character '{character_to_count}' appears {result} times in the file." where:

'{character_to_count}' is the character that the user is counting (the exact character entered by the user).

{result} is the number of occurrences of that character in the file.

If the character is not found, print "Character not found in the file."

Sample test cases :

Input 1 :

test.txt

This is a test file to check the character count.

e

Output 1 :

The character 'e' appears 5 times in the file.

Input 2 :

sample.txt

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

L

Output 2 :

The character 'L' appears 3 times in the file.

Input 3 :

data.txt

Some random content with no special character.

\$

Output 3 :

Character not found in the file.

Input 4 :

document.txt

This document contains multiple spaces and a character count.

Output 4 :

The character ' ' appears 10 times in the file.

Program

```
def count_character_in_file(file_name, character_to_count):
    file = open(file_name, 'w')
    file.write(input_string)
    file.close()

    file = open(file_name, 'r')
    content = file.read()
    file.close()

    count = content.lower().count(character_to_count.lower())
    return count

file_name = input()
input_string = input()
character_to_count = input()
```

week 8 - Exception handling

1. Write a program that calculates the average of a list of integers. The program first prompts the user to enter the length of the list (n), followed by each element of the list. The program should include error handling to ensure that:

The length of the list (n) is a positive integer.

Each input element is a valid integer. If the user enters invalid data at any point, the program should terminate with the appropriate error message. The error handling should cover the following scenarios:

If n is not a valid positive integer, print:

Error: The length of the list must be a non-negative integer.

If a non-numeric value is entered for n, print:

Error: You must enter a numeric value.

If any of the elements in the list are non-numeric, print:

Error: You must enter a numeric value.

If all inputs are valid, the program should calculate the average of the list of integers and print it with two decimal places.

Input format :

The first line of input is an integer n, representing the length of the list. n must be a positive integer.

The next n lines contain one integer per line, representing each element of the list.

Output format :

If the length of the list (n) is not a positive integer or is non-numeric, the program will print:

Error: The length of the list must be a non-negative integer.

or

Error: You must enter a numeric value.

If a non-numeric value is entered for any element of the list, the program will print:

Error: You must enter a numeric value.

If all inputs are valid, the program calculates and prints the average of the integers in the list, rounded to two decimal places:

The average is: <average>

Sample test cases :

Input 1 :

3

1

2

3

Output 1 :

The average is: 2.00

Input 2 :

/

1

3

Output 2 :

Error: You must enter a numeric value.

Input 3 :

0

0

Output 3 :

Error: The length of the list must be a non-negative integer.

Program

```
try:
    n=int(input())
    if n==0:
        print("Error: The length of the list must be a non-negative integer.")
    else:
        sum=0
        for i in range(n):
            v=int(input())
            sum+=v
        print(f"The average is : {(sum/n):.2f}")
except Exception:
    print("Error: You must enter a numeric value.")
```

2. In a voting system, a person must be at least 18 years old to be eligible to vote. If a user enters an age below 18, the system should raise a user-defined exception indicating that they are not eligible to vote.

Input format :

The input contains a positive integer representing age.

Output format :

If the age is less than 18, the output displays "Not eligible to vote".
Otherwise, the output displays "Eligible to vote".

Sample test cases :

Input 1 :

18

Output 1 :

Eligible to vote

Input 2 :

12

Output 2 :

Not eligible to vote

Program

```
try:
    n=int(input())
    if n>= 18:
        print("Eligible to vote")
    else:
        raise ValueError()
except ValueError:
    print("Not eligible to vote")
```

3. Write a program to validate a student's register number and mobile number using user-defined exceptions. The validation should follow these rules:

Register Number Validation:

The register number should be exactly 9 characters.

It must follow this format: 2 digits, followed by 3 letters, and then 4 digits.

For example, a valid register number could be: 12ABC1234.

If the register number fails any of the conditions, raise the following exception:

InvalidRegisterNumberException with the message:

"Register Number should have exactly 9 characters."

"Register Number should have the format: 2 numbers, 3 characters, and 4 numbers."

Mobile Number Validation:

The mobile number should be exactly 10 digits.

It must contain only digits.

If the mobile number fails any of the conditions, raise the following exception:

InvalidMobileNumberException with the message:

"Mobile Number should have exactly 10 characters."

"Mobile Number should only contain digits."

If any of the above conditions fail, the program should throw the appropriate exception and display the specific error message. If both numbers are valid, print "Valid".

Input format :

The first line of the input consists of a string representing the Register number.

The second line of the input consists of a string representing the Mobile number.

Output format :

The output should display any one of the following messages:

If both numbers are valid, print "Valid".

If an exception is raised, print "Invalid with exception message: ", followed by the specific exception message.

Sample test cases :**Input 1 :**

19ABC1001

9949596920

Output 1 :

Valid

Input 2 :

19ABC1001

99495969209

Output 2 :

Invalid with exception message: Mobile Number should have exactly 10 characters.

Input 3 :

19ABC10019

9949596920

Output 3 :

Invalid with exception message: Register Number should have exactly 9 characters.

Input 4 :

195AC1001

9949596920

Output 4 :

Invalid with exception message: Register Number should have the format: 2 numbers, 3 characters, and 4 numbers.

Input 5 :

19ABC1001

994C596920

Output 5 :

Invalid with exception message: Mobile Number should only contain digits.

Program

```
class InvalidRegisterNumberException(Exception):
    pass
class InvalidMobileNumberException(Exception):
    pass

reg_no = input()
mob_no = input()

try:
    if len(reg_no) != 9:
        raise InvalidRegisterNumberException("Register Number should have exactly 9
characters.")

    if not (reg_no[0:2].isdigit() and reg_no[2:5].isalpha() and reg_no[5:9].isdigit()):
        raise InvalidRegisterNumberException("Register Number should have the
format: 2 numbers, 3 characters, and 4 numbers.")

    if len(mob_no) != 10:
        raise InvalidMobileNumberException("Mobile Number should have exactly 10
characters.")

    if not mob_no.isdigit():
        raise InvalidMobileNumberException("Mobile Number should only contain
digits.")

    print("Valid")

except Exception as e:
    print(f"Invalid with exception message: {e}")
```

4. Write a program to obtain the start time and end time for the stage event show. If the user enters a different format other than specified, an exception occurs and the program is interrupted. To avoid that, handle the exception and prompt the user to enter the right format as specified.

Start time and end time should be in the format 'YYYY-MM-DD HH:MM:SS'

If the input is in the above format, print the start time and end time.

If the input does not follow the above format, print "Event time is not in the format "

Input format :

The first line of input consists of the start time of the event.

The second line of the input consists of the end time of the event.

Output format :

If the input is in the given format, print the start time and end time.

If the input does not follow the given format, print "Event time is not in the format".

Sample test cases :**Input 1 :**

2022-01-12 06:10:00

2022-02-12 10:10:12

Output 1 :

2022-01-12 06:10:00

2022-02-12 10:10:12

Input 2 :

2022-01-12 06:10:00

2022-02-12 10:00:

Output 2 :

Event time is not in the format

Input 3 :

2022-01-12 06:10:00

2022-02-31 10:10:12

Output 3 :

Event time is not in the format

Input 4 :

2022-01-12 10:75:00

2022-02-12 10:10:80

Output 4 :

Event time is not in the format

Program

try:

start = input()

end = input()

for time_str in [start, end]:

if len(time_str) != 19:

raise Exception

if time_str[4] != '-' or time_str[7] != '-' or time_str[10] != ' ' or \
time_str[13] != ':' or time_str[16] != ':':

raise Exception

year = int(time_str[0:4])

month = int(time_str[5:7])

day = int(time_str[8:10])

hour = int(time_str[11:13])

minute = int(time_str[14:16])

second = int(time_str[17:19])

if not (1 <= month <= 12): raise Exception

if not (0 <= hour <= 23): raise Exception

if not (0 <= minute <= 59): raise Exception

if not (0 <= second <= 59): raise Exception

days_list = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):

days_list[1] = 29

if not (1 <= day <= days_list[month - 1]):

raise Exception

print(start)

print(end)

except:

print("Event time is not in the format")

5. Helen is an adventurous explorer on a quest to discover the hidden treasures marked on an ancient enchanted map. To unlock the map's secrets, you need to input two pairs of coordinates (x1, y1) and (x2, y2) as directed by the mystical map.

The enchanted map is guarded by the "Guardian of Arguments," a magical guardian who enforces the map's rules. The correct coordinates reveal the sum of squares leading to the treasure's location. Incorrect inputs warn adventurers.

Prompt the user to enter four integer coordinates separated by spaces in a single line.

Split the input string into an array of arguments and check if exactly four coordinates are provided. If not, throw a custom exception named CheckArgument with the message "Guardian of Arguments awakens!"

If four coordinates are provided, parse them as integers, calculate the sum of the squares of these coordinates, and display the sum of squares as the treasure's secret location.

If an incorrect number of coordinates is provided or if there's an issue with parsing, catch the CheckArgument exception, and display "Be aware!" followed by the exception message.

Use a finally block to close any resources used.

Input format :

The input consists of 4 space-separated integers representing the coordinates (x1, y1) and (x2, y2).

Output format :

If the input is valid, the output prints "Treasure's Secret Location: " followed by an integer representing the location of the treasure.

Otherwise, the output prints "Be aware!"

Guardian of Arguments awakens!" each in a new line.

Sample test cases :

Input 1 :

1 -2 -3 4

Output 1 :

Treasure's Secret Location: 30

Input 2 :

25 12 -98

Output 2 :

Be aware!

Guardian of Arguments awakens!

Program

```
class CheckArgument(Exception):
    pass

try:
    user_input = input().split()
    if len(user_input) != 4:
        raise CheckArgument("Guardian of Arguments awakens!")
    coords = [int(x) for x in user_input]

    sum_squares = sum(c**2 for c in coords)
    print(f"Treasure's Secret Location: {sum_squares}")

except (CheckArgument, ValueError):
    print("Be aware!")
    print("Guardian of Arguments awakens!")
```

Week 9 & 10 - Pandas

1. Ram is working on a project to validate voter IDs for a government election system. He seeks your help in writing a program that takes user input for voter IDs, processes the data, and outputs a data frame with the voter IDs using Pandas.

Input format :

The input consists of a space-separated string representing voter IDs.

Output format :

The output displays a data frame containing the voter IDs.

Sample test cases :

Input 1 :

V12345 V23456 V34567 V45678 V56789

Output 1 :

```
Voter ID
0  V12345
1  V23456
2  V34567
3  V45678
4  V56789
```

Program

```
import pandas as pd
user_input = input().split()
df = pd.DataFrame(user_input, columns=['Voter ID'])
print(df)
```

2. Susi is the HR manager at a tech company, and she needs to streamline the employee onboarding process.

She wants to create a program that takes input from new employees and stores their details in a pandas data frame for easy management.

The program should handle information such as name, age, qualification, salary, and performance.

Input format :

The first line consists of a string representing the employee's name.

The second line consists of an integer representing the employee's age.

The third line consists of a string representing the employee's educational qualification.

The fourth line consists of a float representing the employee's salary.

The fifth line consists of a string representing the employee's performance level.

Output format :

The program outputs a single row DataFrame with columns representing the personal information provided:

'Name': Name of the individual

'Age': Age of the individual

'Qualification': Qualification of the individual

'Salary': Salary of the individual

'Performance': Performance rating of the individual

The data frame contains one row corresponding to the provided personal information.

Sample test cases :

Input 1 :

John Doe

28

Bachelor's Degree

50000.0

Excellent

Output 1 :

	Name	Age	Qualification	Salary	Performance
0	John Doe	28	Bachelor's Degree	50000.0	Excellent

Program

```
import pandas as pd
```

```
name = input()
age = int(input())
qualification = input()
salary = float(input())
performance = input()
```

```
data = {
    'Name': [name],
    'Age': [age],
    'Qualification': [qualification],
    'Salary': [salary],
    'Performance': [performance]
}
```

```
df = pd.DataFrame(data)
print(df)
```

3. John is a teacher who wants to keep track of his five students' information including their names, ages, cities, qualifications, and marks in different subjects.

He needs a program to input this information, calculate the average marks of each student and output displays the data frame format using pandas.

Help John by writing a program to accomplish this task.

Input format :

The input consists of 5 sets of lines, each representing information about a student.

Each set contains two lines:

The first line contains the student's name as a string, age as an integer, city as a string, and qualification as a string separated by space.

The second line contains space-separated integer values representing the marks obtained by the student in different subjects.

Output format :

The output should display a data frame containing the following columns:

'Name': Name of the student

'Age': Age of the student

'City': City of the student

'Qualification': Qualification of the student

'Average Mark': Average marks obtained by the student, rounded to 2 decimal places.

Sample test cases :**Input 1 :**

Alice 25 NewYork PhD

90 85 92 88 94

Bob 22 LosAngeles Master

78 80 75 82 79

Carol 28 Chicago Bachelor

65 70 68 72 69

Dave 23 SanFrancisco Bachelor

85 88 82 90 87

Eve 27 Miami Master

93 89 94 91 96

Output 1 :

	Name	Age	City	Qualification	Average Mark
0	Alice	25	NewYork	PhD	89.80
1	Bob	22	LosAngeles	Master	78.80
2	Carol	28	Chicago	Bachelor	68.80
3	Dave	23	SanFrancisco	Bachelor	86.40
4	Eve	27	Miami	Master	92.60

Program

```
import pandas as pd
```

```
names = []
```

```
ages = []
```

```
cities = []
```

```
qualifications = []
```

```
averages = []
```

```
for _ in range(5):
```

```
    personal_info = input().split()
```

```

names.append(personal_info[0])
ages.append(int(personal_info[1]))
cities.append(personal_info[2])
qualifications.append(personal_info[3])

marks = list(map(int, input().split()))
avg = sum(marks) / len(marks)
averages.append(f"{avg:.2f}")

data = {
    'Name': names,
    'Age': ages,
    'City': cities,
    'Qualification': qualifications,
    'Average Mark': averages
}

df = pd.DataFrame(data)
print(df)

```

4. Sindhu, an analyst, is responsible for analyzing the performance data of two teams, Team A and Team B, over multiple matches. The performance scores for each team are stored in separate series (Series 1 and Series 2).

She needs to develop a program that takes the performance scores of both teams for a given match. Finally, Sindhu needs to concatenate these series into a single series for further analysis.

Let's help Sindhu write this program.

Input format :

The first line consists of space-separated integers representing the performance scores of Team A (Series 1).

The second line consists of space-separated integers representing the performance scores of Team B (Series 2).

Output format :

The output consists of three parts:

The performance scores of Team A displayed as a pandas Series. This is denoted by the statement: "Series 1:" followed by the pandas Series representation, including the dtype.

The performance scores of Team B displayed as a pandas Series. This is denoted by the statement: "Series 2:" followed by the pandas Series representation, including the dtype.

The concatenated Series of performance scores of both teams. This is denoted by the statement: "After concatenating:" followed by the concatenated pandas Series representation, including the dtype.

Sample test cases :

Input 1 :

```
1 2 3 4
5 6 7 8
```

Output 1 :

Series 1:

```
0    1
1    2
2    3
3    4
```

dtype: int64

Series 2:

```
0    5
1    6
2    7
3    8
```

dtype: int64

After concatenating:

```
0    1
1    2
2    3
3    4
0    5
1    6
2    7
3    8
```

dtype: int64

Program

```
import pandas as pd

team_a_scores = list(map(int, input().split()))
team_b_scores = list(map(int, input().split()))

series1 = pd.Series(team_a_scores)
series2 = pd.Series(team_b_scores)

concatenated_series = pd.concat([series1, series2])

print("Series 1:")
print(series1)
print("Series 2:")
print(series2)
print("After concatenating:")
print(concatenated_series)
```

5. Jinu needs a program to collect performance data from Team A and Team B for multiple matches. Each match consists of two sets of integer scores, one for Team A and the other for Team B. The program should prompt Jinu to input these scores for each match.

After collecting the data, the program should display the performance scores of both teams for each match in separate data frames. Additionally, it should concatenate the DataFrames for further analysis.

Input format :

The user is prompted to input the performance scores of Team A and Team B for each match.

Each match's input consists of four lines, where each line contains two space-separated integers representing the performance scores for Team A and Team B in that match.

Each integer represents the performance score of a team in a particular aspect of the match.

Output format :

The program outputs two DataFrames (df1 and df2) representing the performance scores of Team A and Team B for each match, respectively.

Each DataFrame has two columns labeled 'A' and 'B', where 'A' contains the performance scores of Team A and 'B' contains the performance scores of Team B.

After concatenating the DataFrames, the program outputs the concatenated DataFrame.

The concatenated DataFrame includes keys 'key1' and 'key2' to identify the source of data for each match.

Sample test cases :

Input 1 :

```
1 2
7 8
6 7
3 4
1 9
1 3
7 8
1 0
```

Output 1 :

df1:

```
  A B
0 1 2
1 7 8
2 6 7
3 3 4
```

df2:

```
  A B
0 1 9
1 1 3
2 7 8
3 1 0
```

After concatenating:

```
      A B
key1 0 1 2
      1 7 8
      2 6 7
      3 3 4
key2 0 1 9
      1 1 3
      2 7 8
      3 1 0
```

Program

```
import pandas as pd
```

```
team_a_df1 = []
```

```
team_b_df1 = []
```

```
for _ in range(4):
```

```
    a, b = map(int, input().split())
```

```
    team_a_df1.append(a)
```

```
    team_b_df1.append(b)
```

```
team_a_df2 = []
```

```
team_b_df2 = []
```

```
for _ in range(4):
```

```
    a, b = map(int, input().split())
```

```
    team_a_df2.append(a)
```

```
    team_b_df2.append(b)
```

```
df1 = pd.DataFrame({'A': team_a_df1, 'B': team_b_df1})
```

```
df2 = pd.DataFrame({'A': team_a_df2, 'B': team_b_df2})
```

```
concatenated_df = pd.concat([df1, df2], keys=['key1', 'key2'])
```

```
print("df1:")
```

```
print(df1)
```

```
print("df2:")
```

```
print(df2)
```

```
print("After concatenating:")
```

```
print(concatenated_df)
```

```
=====
```